

# spinalSynth: Testing Specification

This document provides the formal specifications for the simulation test suite in `spinalSynth`. It defines the verification logic, simulated environments, stimulus parameters, and assertion constraints for all unit and integration testbenches.

## 1. Unit Tests

This chapter contains the specifications for testing individual, isolated hardware modules.

### 1.1 Attenuator Unit Test (`AttenuatorSim`)

#### Purpose

Verifies the custom DSP volume attenuator module (`Attenuator`) for mathematical precision, pipeline latency, and reset stability.

#### Simulated Environment

- Component Under Test:** `Attenuator`
- Clock Domain:** 24 MHz synchronous master clock (simulation period = 10 units).
- Reset:** Asynchronous, Active-High.

#### Input Stimulus & Signals

- `io.sampleIn`: `Flow[SInt]` (16 bits)
- `io.volume`: `UInt` (8 bits)
- `io.sampleOut`: `Flow[SInt]` (16 bits)

#### Test Cases

##### 1.1.1 Reset Stability

- Action:** Assert `reset` high for 5 clock cycles while driving random values on `io.sampleIn.payload` and `io.volume`.
- Assertion:** Verify that `io.sampleOut.payload` is strictly held at 0 and `io.sampleOut.valid` is strictly `False`.

##### 1.1.2 Mathematical Scaling

- Action:** Pulse `io.sampleIn.valid` high for 1 clock cycle with a specific payload and volume, then return it to `False`. Wait 1 clock cycle for the pipeline register.
- Assertion:** Verify that `io.sampleOut.valid` is `True` during the second cycle and its payload matches the expected fractional attenuation exactly: 
$$\text{expected} = \frac{\text{sampleIn.payload}}{\text{volume} + 1}$$
- Test Vectors:**

| Input Sample | Volume | Expected Output | Notes                                     |
|--------------|--------|-----------------|-------------------------------------------|
| 20000        | 255    | 19921           | Full scale fractional (19921.8 truncated) |
| 20000        | 128    | 10000           | Half scale (Exact)                        |
| -20000       | 64     | -5000           | Quarter scale negative                    |
| -32768       | 0      | 0               | Silent (Mute)                             |

##### 1.1.3 Cycle-Accurate Latency & Pipelining

- Action:** Pulse `io.sampleIn.valid` back-to-back for 3 consecutive clock cycles with distinct samples:
  - Cycle 1: Sample = 10000, Volume = 255
  - Cycle 2: Sample = 20000, Volume = 128
  - Cycle 3: Sample = -10000, Volume = 64
- Assertion:** Verify that:
  - Cycle 1: `sampleOut.valid` is `False`.
  - Cycle 2: `sampleOut.valid` is `True`, payload = 9960.
  - Cycle 3: `sampleOut.valid` is `True`, payload = 10000.
  - Cycle 4: `sampleOut.valid` is `True`, payload = -2500.
  - Cycle 5: `sampleOut.valid` is `False`.
  - This confirms the 1-cycle pipeline throughput operates continuously without stalls or register leakage.

## 1.2 Register Bank Unit Test (RegisterBankSim)

### Purpose

Verifies the parameter storage register bank module (RegisterBank) for reset defaults, single-byte register updates, and atomic 24-bit frequency word commitment.

### Simulated Environment

- **Component Under Test:** RegisterBank
- **Clock Domain:** 24 MHz master clock (simulation period = 10 units).
- **Reset:** Asynchronous, Active-High.

### Input Stimulus & Signals

- io.regWrite: Flow[RegisterWrite] (containing 8-bit address and 8-bit data)
- io.config: OscillatorConfig (output bundle containing freqWord, waveSelect, pwmWidth, volume)

### Test Cases

#### 1.2.1 Reset Defaults

- **Action:** Start simulation with reset asserted.
- **Assertion:** Verify that all output configuration fields in io.config are held strictly at 0.

#### 1.2.2 Single-Byte Direct Updates

- **Action:** Write individually to non-atomic registers using the io.regWrite port:
  - Write 0x03 (address 0x03 - WAVE\_SEL)
  - Write 0xA5 (address 0x04 - PWM\_WIDTH)
  - Write 0x7F (address 0x05 - VOLUME)
- **Assertion:** Verify that the corresponding output fields (config.waveSelect, config.pwmWidth, and config.volume) are updated to the written values on the next clock cycle.

#### 1.2.3 Atomic 24-Bit Frequency Commitment

- **Action:** Perform a sequential write sequence to verify shadow staging and atomic trigger commitment:
  1. Write 0x55 to 0x00 (FREQ\_LOW).
    - **Assertion:** Verify that config.freqWord remains unchanged (stages in shadow register).
  2. Write 0xAA to 0x01 (FREQ\_MID).
    - **Assertion:** Verify that config.freqWord remains unchanged (stages in shadow register).
  3. Write 0x0C to 0x02 (FREQ\_HIGH).
    - **Assertion:** Verify that on the next clock cycle, the active output config.freqWord updates atomically to 0x0CAA55 (830037 in decimal) all at once.

---

## 1.3 UART Protocol Decoder Unit Test (UartProtocolDecoderSim)

### Purpose

Verifies the FSM protocol parser (UartProtocolDecoder) for reset safety, successful command framing, byte-spacing delay tolerance, and command-valid bounds.

### Simulated Environment

- **Component Under Test:** UartProtocolDecoder
- **Clock Domain:** 24 MHz master clock (simulation period = 10 units).
- **Reset:** Asynchronous, Active-High.

### Input Stimulus & Signals

- io.rxByte: Flow[Bits] (input stream representing incoming serial bytes)

- `io.regWrite:Flow[RegisterWrite]` (output transaction flow)

Test Cases

1.3.1 Reset Safety

- **Action:** Assert reset while pulsing `rxByte.valid` with random bytes.
- **Assertion:** Verify that `io.regWrite.valid` remains strictly `False`.

1.3.2 Valid Command Framing (3-Byte Stream)

- **Action:** Push three bytes back-to-back:
  - Byte 1: `0x01` (WriteRegister command)
  - Byte 2: `0x02` (`FREQ_HIGH` address)
  - Byte 3: `0xAB` (Register data payload)
- **Assertion:** Verify that:
  - `regWrite.valid` is `False` after Byte 1 and Byte 2.
  - `regWrite.valid` is `True` in the exact cycle Byte 3 is pushed, and its payload contains `address = 0x02` and `data = 0xAB`.
  - `regWrite.valid` drops back to `False` in the following cycle.

1.3.3 Byte-Spacing Delay Tolerance

- **Action:** Push a 3-byte transaction with arbitrary timing spacing to simulate slow UART transmissions:
    - Push Byte 1 (`0x01`), then wait 25 clock cycles.
    - Push Byte 2 (`0x05 - VOLUME`), then wait 50 clock cycles.
    - Push Byte 3 (`0x7F`), then wait 1 clock cycle.
  - **Assertion:** Verify that the FSM maintains state synchronization, and atomically asserts `regWrite.valid = True` with `address = 0x05` and `data = 0x7F` exactly when Byte 3 is pushed.
- 

1.4 Timing Generator Unit Test (`TimingSim`)

Purpose

Verifies the master clock divider module (`TimingGenerator`) for reset safety, cycle-accurate tick intervals (`480\text{ kHz}`) phase ticks and (`48\text{ kHz}`) sample ticks, and strict synchronous alignment between clock domains.

Simulated Environment

- **Component Under Test:** `TimingGenerator`
- **Clock Domain:** 24 MHz master clock (simulation period = 10 units).
- **Reset:** Asynchronous, Active-High.

Input Stimulus & Signals

- `io.phaseTick: Bool` (output divider pulse every 50 master clock cycles)
- `io.sampleTick: Bool` (output divider pulse every 500 master clock cycles)

Test Cases

1.4.1 Reset Stability

- **Action:** Assert active-high reset for 20 clock cycles.
- **Assertion:** Verify that both `phaseTick` and `sampleTick` remain strictly `False`.

1.4.2 Interval Precision & Long-Term Stability

- **Action:** Run the simulation and measure consecutive tick intervals after deasserting reset:
  - Track 10 consecutive `phaseTick` events.
  - Track 5 consecutive `sampleTick` events.
- **Assertion:** Assert that every `phaseTick` interval is exactly 50 clock cycles and every `sampleTick` interval is exactly 500 clock cycles, ensuring zero clock drift.

1.4.3 Synchronous Domain Alignment

- **Action:** Monitor outputs over 5 full sample cycles (\$2500\$ clock cycles).
  - **Assertion:** Verify that *every single time* `sampleTick` is asserted (`True`), `phaseTick` is also synchronously asserted (`True`).
- 

1.5 Waveform Generator Unit Test (`WaveformSim`)

Purpose

Verifies the digital oscillators core module (`Generators`) for mathematical wave formatting precision (Sawtooth, Square, Triangle) and pulse-width comparator thresholds (PWM) across boundary phases.

Simulated Environment

- **Component Under Test:** `Generators`
- **Clock Domain:** None (Combinational Verification).

Input Stimulus & Signals

- `io.phase`: `UInt` (24 bits)
- `io.pwmWidth`: `UInt` (8 bits)
- `io.waves`: `Waveforms` (output bundle containing `saw`, `square`, `pwm`, `tri`)

Test Cases

1.5.1 Peak & Zero-Crossing Waveform Math

- **Action:** Drive specific static phase angles and assert output bounds:
  - **Phase `0x000000` (Start):** Assert `saw` = -32768, `square` = -32768, `tri` = -32768.
  - **Phase `0x400000` (1/4 Cycle):** Assert `saw` = -16384, `square` = -32768, `tri` = 0.
  - **Phase `0x7FFFFFFF` (Pre-Half Transition):** Assert `saw` = -1, `square` = -32768, `tri` = 32767.
  - **Phase `0x800000` (Half Cycle / Toggle):** Assert `saw` = 0, `square` = 32767, `tri` = 32767.
  - **Phase `0xC00000` (3/4 Cycle):** Assert `saw` = 16384, `square` = 32767, `tri` = -1.
  - **Phase `0xFFFFF` (Wrap Boundary):** Assert `saw` = 32767, `square` = 32767, `tri` = -32768.

1.5.2 PWM Comparator Threshold Logic

- **Action:** Set pulse duty configurations and verify switching thresholds:
    - Set `pwmWidth` = `0x80` (50%): Verify `pwm` = 32767 at phase `0x7FFFFFFF` and `pwm` = -32768 at phase `0x800000`.
    - Set `pwmWidth` = `0x40` (25%): Verify `pwm` = 32767 at phase `0x3FFFFFFF` and `pwm` = -32768 at phase `0x400000`.
- 

1.6 I2S Transmitter Unit Test (`I2STransmitterSim`)

Purpose

Verifies the I<sup>2</sup>S serial audio transmitter module (`I2STransmitter`) for startup safety, idle low power states, cycle-accurate timing patterns (according to the modulation table), and correct parallel-to-serial data conversion.

Simulated Environment

- **Component Under Test:** `I2STransmitter`
- **Clock Domain:** 24 MHz master clock (simulation period = 10 units).
- **Reset:** Asynchronous, Active-High.

Input Stimulus & Signals

- `io.sampleIn`: `Flow[SInt]` (16 bits, incoming parallel samples)
- `io.bclk`: `Bool` (output continuous bit clock)
- `io.lrcclk`: `Bool` (output left/right word select channel line)
- `io.sdata`: `Bool` (output serialized data line)

Test Cases

1.6.1 Reset Stability & Idle Power State

- **Action:** Assert reset for 20 clock cycles, deassert, and run for 20 more cycles without pulsing `sampleIn.valid`.
- **Assertion:** Verify that throughout this duration:
  - `bclk` is held strictly `False`.
  - `lrcclk` is held strictly `True` (standard passive high line state).
  - `sdata` is held strictly `False`.

1.6.2 Serial Word Bitstream Precision

- **Action:** Pulse `sampleIn.valid = True` for 1 cycle loading sample `0xA5A5` (binary `1010010110100101`).
- **Assertion:** Align with the start of the serialization frame (`lrcclk` goes `False` for Left channel), and sample `sdata` at each bit's middle interval (when `bclk` is `True`):
  - Verify that the stream of 16 sequential bits matches the MSB-first bits of `0xA5A5` perfectly.

1.6.3 Timing Pattern & Frame Duration

- **Action:** Wait for `lrcclk` to toggle, then measure the individual bit clock intervals and the full Left-to-Right frame duration.
- **Assertion:** Verify that:
  - The first 8 bit intervals match the cycle-accurate modulation pattern: 16, 16, 15, 16, 16, 15, 16, 15 clock cycles.
  - The full Left/Right stereo frame completes in exactly 500 clock cycles ( $\frac{48}{\text{kHz}}$  sampling rate).

2. Integration Tests

This chapter contains the specifications for verifying multi-module and full-system interactive integration behavior.

2.1 Complete System Integration Test (`SynthSim`)

Purpose

Verifies the end-to-end synthesizer system (`Synth`) for seamless hardware module cooperation, including UART reception, register bank parsing, dynamic digital sound wave synthesis (PWM), volume scaling, and stereo I<sup>2</sup>S serial streaming in real-time.

Simulated Environment

- **Component Under Test:** `Synth`
- **Clock Domain:** 24 MHz master clock (simulation period = 10 units).
- **Reset:** Asynchronous, Active-High.

Input Stimulus & Signals

- `io.uartRx`: `Bool` (input UART serial receive line)
- `io.i2sBclk`: `Bool` (output I2S bit clock)
- `io.i2sLrclk`: `Bool` (output I2S left/right channel select)
- `io.i2sData`: `Bool` (output I2S serial audio data stream)

Test Cases

2.1.1 Power-On Silence Idle

- **Action:** Deassert reset and run simulation for 1000 clock cycles with `uartRx` held idle (`True`).
- **Assertion:** Verify that the generated left/right stereo I2S samples are strictly silent (`0`), confirming the attenuator starts safely muted.

2.1.2 Real-time UART Parameter Modulation

- **Action:** Stream a live byte sequence over the `uartRx` line at 115200 Baud (208 master cycles per bit) to dynamically configure the synthesizer:
  1. Write `0x02` (PWM mode) to Waveform Select (`0x03`).
  2. Write `0x80` (50% Duty cycle) to PWM Width (`0x04`).
  3. Write `0xFF` (Max Volume) to Volume (`0x05`).
  4. Write atomic DDS Frequency tuning word `0x080000` (Low = `0x00`, Mid = `0x00`, High = `0x08` commit to target  $\frac{15}{\text{kHz}}$ ).

- **Assertion:** Capture 25 I2S frames and verify:
  - **Stereo Alignment:** Left and Right samples are perfectly identical for all frames.
  - **Dynamic Audio Response:** Outputs are no longer silent (non-zero PWM waves are generated).
  - **DSP Correctness:** Non-zero sample values transition perfectly between peak positive (32639) and peak negative (-32640) values, demonstrating a true 50% duty-cycle PWM square wave swinging at  $15\text{ kHz}$ .